2027 IEEE International Symposium on High-Performance Computer Architecture (HPCA)

DiVA: Algorithm-System Co-Design for Visual Autoregressive Models via Submap-Level Retrievals

Jain Koo†, Sungbin Kim†, Enhyeok Jang†, Sungwoo Kim†, Yingyan (Celine) Lin‡, and Won Woo Ro†
†Yonsei University ‡Georgia Institute of Technology
{jain.koo, sungbin.kim, enhyeok.jang, sungwoo.kim, wro}@yonsei.ac.kr, celine.lin@gatech.edu

***Abstract*—Visual autoregressive (VAR) models utilize coarse-to-fine next-resolution prediction and have emerged as promising alternatives to diffusion models for image generation. Unlike LLMs, VAR generates multiple query tokens at each decoding stage, causing the KV cache to grow rapidly across scales and making inference challenging under limited GPU memory capacity. While prior KV cache retrievals reduce memory burden by offloading the full cache and selectively fetching critical entries, we note that they become suboptimal for VAR in both latency and accuracy, since the criticality estimation process fundamentally changes. To address this, we present DiVA, an algorithm–system co-designed architecture for KV retrieval in VAR models. DiVA is built on S2MR, which shifts the granularity of KV retrieval from tokens to submaps. S2MR uses submap-representative scoring, compressing each key submap into a representative vector to reduce computational overhead while preserving accuracy. To leverage these benefits, DiVA integrates dedicated hardware for submap compression, row-wise scoring, and normalization. DiVA achieves up to 4.23× speedup over the baselines and improves energy efficiency by up to 3.02×, with only 2.7% area overhead.**

I. Introduction

Visual autoregressive (VAR) models [15], [69] have emerged as promising alternatives to diffusion models [35], [42] in image generation, due to their scalability and generality. Unlike conventional autoregressive (AR) models [12], [52], which generate text or images in a next-token prediction manner by sequentially producing a single token at each stage, VAR adopts a coarse-to-fine next-resolution prediction paradigm. In each stage, VAR generates an entire token map at a target resolution, with all tokens within the token map produced in parallel. Thus, the number of decoding steps for high-resolution generation is greatly reduced.

However, this design fundamentally changes the inference characteristics of Transformer-based generation [10], [11], [42] compared with conventional large language models (LLMs) and AR models. As shown in Fig. 1, VAR generates a large number of visual tokens in parallel at each decoding step, and its sequence length grows rapidly as the target resolution increases across scales. Accordingly, unlike next-token AR models [18], [24], [29], [38], [50], [65], [67], where the KV cache size increases linearly [8], [16], [36], [51], [71] as each decoding step appends a single token, VAR appends a large number of KV entries at every step, causing the KV cache size to grow much more rapidly during inference. The resulting KV cache footprint places pressure on GPU memory capacity and makes the practical deployment of large-scale VAR models challenging.

To address the tremendous KV cache size, prior works [22], [30], [37] have explored KV cache retrieval systems that offload the full cache to CPU memory or storage and selectively fetch only critical KV entries [39], [40], [55], [59]. By reducing the size of KV data fetched to the GPU, these systems effectively lower GPU memory usage while preserving generation quality. While these are effective for LLMs, we find that existing KV cache retrieval systems become suboptimal for VAR in terms of both latency and accuracy, as VAR fundamentally changes the criticality estimation process, which evaluates the importance of each KV entry.

During LLM decoding, each step involves a single query vector, and criticality estimation is therefore performed as a GEMV operation. Existing retrieval systems were designed to reduce this GEMV overhead using approximation techniques such as channel reduction [30] and token pooling [20], [54], which reduce the feature dimension or the number of tokens involved in criticality estimation. In contrast, VAR processes a large number of query tokens simultaneously, expanding the same criticality estimation process from a GEMV into a large GEMM operation. Consequently, even after applying these approximations, criticality estimation becomes the dominant performance bottleneck in VAR, accounting for more than 70% of the runtime at later scales, as shown in Fig. 3.

Moreover, these systems also introduce accuracy challenges when applied to VAR, since each query exhibits a distinct importance distribution. Specifically, important channel indices can differ substantially across rows of the query matrix, while important key tokens can vary across queries of the attention-score matrix. As a result, channel reduction may fail to preserve the critical channels of individual queries, whereas query-key token pooling can blur the distinct key importance of each query, leading to substantial accuracy degradation. These observations reveal that existing KV retrievals are unsuitable for VAR, motivating a new retrieval algorithm tailored to VAR's multi-token generation.

Based on these observations, we propose DiVA, an algorithm-system co-designed architecture for efficient and accurate KV cache retrieval in VAR models. At the algorithm level, DiVA introduces S2MR, a retrieval method specifically designed for VAR's next-resolution prediction paradigm. Unlike conventional retrieval methods that select individual KV tokens, S2MR elevates the retrieval granularity from tokens to submaps. A submap is a contiguous group of KV tokens generated together at the same resolution scale, which collectively forms a complete spatial token map of the image at that scale. Thus, submap-level retrieval may be better suited to preserving image fidelity, but token-level retrieval may retain only fragmented portions of each spatial token map, disrupting its structural completeness and leading to a drop in accuracy.

To validate this, we analyze the attention patterns of VAR together with the intra- and inter-submap variation in attention scores, as shown in Fig. 6 and 7. Fig. 6 shows a multi-diagonal attention pattern aligned with stage-wise submap boundaries, suggesting that attention is structured around submaps rather than a single token subset. Consistent with this, Fig. 7 shows that tokens within the same submap exhibit similar attention scores, whereas score differences are larger across submaps, indicating the presence of submap coherence. Thus, submap-level retrieval selects important tokens at the granularity of complete, spatially organized token maps, avoiding the fragmentation that may arise from prior finer methods.

Leveraging this submap coherence, S2MR introduces submap-representative scoring, which represents each submap with a single representative vector during criticality estimation. By evaluating importance at the submap level rather than computing scores for all individual KV entries, S2MR substantially reduces the criticality estimation complexity. Moreover, because each submap is stored as a contiguous KV chunk, S2MR avoids the irregular memory access and sparse gathering overhead that arise in prior finer-granularity retrieval methods. To fully exploit the benefits of S2MR, we design DiVA, a specialized hardware system that efficiently supports S2MR. DiVA consists of a Submap Compressor, a Row Processing Unit, and a bit-level Normalization Unit, and efficiently executes S2MR through a submap-stationary dataflow and channel-wise tiled processing. With this design, the proposed DiVA system effectively eliminates the criticality estimation bottleneck by leveraging the benefits of S2MR, achieving higher performance than the baselines.

In summary, our main contributions are as follows:

- We observe that existing KV retrieval strategies are sub-optimal for VAR because of VAR's inherent multi-query token generation and row-wise divergence in channel and token importance.
- We propose S2MR, a KV retrieval algorithm for VAR that converts the retrieval granularity from tokens to submaps and greatly reduces the criticality estimation overhead.
- We also propose the co-designed DiVA system that efficiently supports our algorithm with specialized engines and execution flow.
- DiVA achieves up to 4.23× speedup, 3.02× improvement in energy efficiency over baselines, and higher accuracy with only 2.7% chip-area overhead.

II. Background

A. Visual Autoregressive Modeling

Conventional AR models [4], [7], [26], [27], [45], [60], [63] rely on next-token prediction, which inevitably leads to

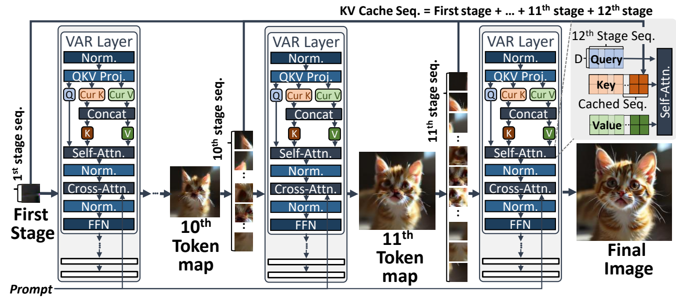


Fig. 1: Overall inference process of the VAR model.

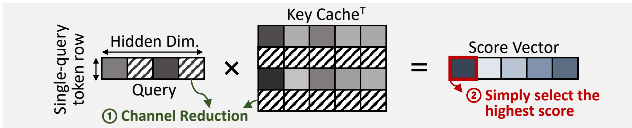
a large number of decoding steps. This becomes severe for high-resolution image synthesis, where the number of tokens scales with spatial resolution, leading to a large number of autoregressive iterations and high end-to-end latency. In contrast, VAR [15], [53], [56], [58] shifts the generation paradigm from next-token prediction to next-resolution prediction. Motivated by the coarse-to-fine nature of visual content, VAR progressively predicts higher-resolution visual representations (i.e., token maps) based on previously generated lower-resolution ones, reducing the number of autoregressive iterations required for high-resolution synthesis as shown in Fig. 1. VAR decodes an image by a sequence of multi-scale token maps, $\mathbf{R} = (\mathbf{r}_1, \ldots, \mathbf{r}_K)$, where the spatial resolution increases with the scale index k, and $\mathbf{r}_K$ corresponds to the token map of the highest-resolution stage. During inference, when predicting the k^{th} token map $\mathbf{r}_k \in [V]^{H_k \times W_k}$, VAR generates all $H_k \times W_k$ tokens in parallel within the k-th stage, enabling the simultaneous generation of thousands of tokens (e.g., 4096 tokens at the 12^{th} stage in Infinity-8B). This multi-token generation property distinguishes VAR from conventional AR models such as LLMs [4], [14], [26], [27], [45], [46], which typically generate a single token per decoding step.

While this design reduces the number of autoregressive iterations, it introduces distinct inference challenges. VAR still relies on attention over long sequences for high-resolution generation, and it does not use causal masking within each token map, increasing the attention computation for a given sequence length. Moreover, VAR's KV cache grows much faster than in next-token prediction models, as each stage appends all tokens within an entire token map rather than a single token. Thus, VAR faces substantial attention latency and KV cache memory pressure during high-resolution generation.

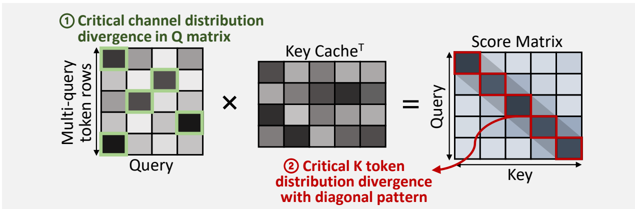
B. KV Cache Retrieval Systems

To reduce high attention computation cost and large KV cache footprints, recent works [5], [32], [43], [62], [70] have primarily focused on reducing the KV cache size [1], [2], [21], [25], [33], [47], [61], [64], [66]. A straightforward approach is KV cache compression, such as eviction or quantization, which reduces memory usage and attention computation but can introduce irreversible information loss and degrade accuracy.

Another approach is *KV cache retrieval* [22], [30], which preserves the full KV cache in larger memory, such as CPU



(a) Criticality estimation in LLM decoding stages.



(b) Criticality estimation in VAR inference stages.

Fig. 2: Comparison of the criticality estimation process in (a) LLM and (b) VAR. Darker colors indicate more critical channels/score elements. The diagonal pattern shown in this example reflects a representative attention head pattern observed in VAR, as illustrated in Fig. 6.

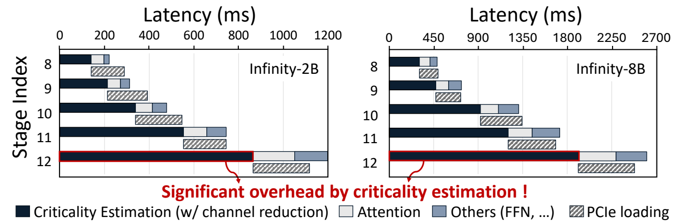


Fig. 3: Latency breakdown of prior retrieval systems in Infinity-2B and Infinity-8B with batch size 1. We use the NVIDIA A6000 GPU for evaluation. Three optimization strategies are applied: (1) query channel reduction (30% of the channels are retained), (2) fetching only 30% of the critical KV cache, and (3) layer-wise KV cache prefetching.

DRAM [40], [55], [59], and selectively transfers only critical KV entries to GPUs, avoiding the permanent information loss of compression. By restricting attention to this subset, they reduce both computation and memory overhead while largely preserving generation quality and enabling scaling beyond GPU memory capacity. But it introduces a new bottleneck due to the limited PCIe bandwidth, as on-demand KV retrieval can expose data movement latency on the critical path and limit inference performance. To mitigate this, prior works incorporate layer-wise KV prefetching to overlap PCIe transfers with GPU computation, hiding latency from critical paths.

To make KV cache retrieval practical under limited PCIe bandwidth, two key design objectives must be satisfied. First, the criticality estimation stage, which determines the importance of KV entries, must be efficient. As this stage is introduced in addition to the original inference pipeline, it should incur minimal overhead so that selected KV entries can be prefetched to the GPU in time. Second, the criticality estimation must be accurate, since selecting unimportant KV entries can lead to generation quality degradation. Inaccurate estimation often necessitates fetching a larger number of KV entries to maintain quality, which in turn increases PCIe traffic and diminishes the effectiveness of latency hiding through prefetching. Prior works have successfully satisfied these requirements in conventional AR models. However, their effectiveness may not directly carry over to VAR due to its multi-token decoding behavior. This difference may introduce additional challenges for the efficiency and accuracy of criticality estimation in KV retrieval systems.

III. Motivation

A. Latency Challenges of KV Retrieval in VAR

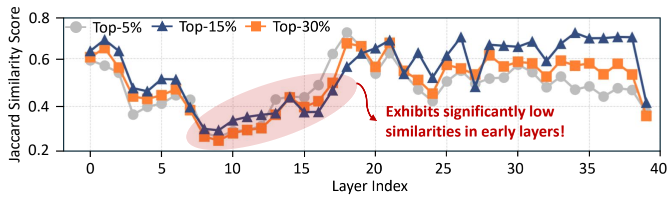
In conventional LLMs [19], [23], [24], [29], [31], [44], [57], each decoding step generates only a single query token, allowing critical KV entries to be identified through a GEMV operation between the query vector and the cached key matrix, as shown in Fig. 2a. Although this GEMV operation introduces non-negligible overhead, prior KV cache retrieval methods effectively reduce its cost through optimization techniques such as channel reduction [30] and token pooling [20]. Channel reduction retains only a subset of high-magnitude feature dimensions, whereas token pooling aggregates multiple query and key tokens into coarse representatives to reduce the effective sequence dimensions. These techniques decrease the feature and sequence dimensions of criticality estimation, respectively, making the overhead manageable in single-query LLM decoding. However, this overhead is significantly amplified in VAR due to its multi-query generation pattern.

Each VAR decoding step generates $H_k \times W_k$ query tokens in parallel, fundamentally changing the criticality estimation. Instead of executing a single query through GEMV, VAR multiplies an entire query matrix against the cached key matrix, transforming the operation into a large GEMM, as shown in Fig. 2b. Even after applying optimizations developed for prior KV cache retrievals, criticality estimation latency increases substantially as its cost scales with the number of query tokens generated at each scale, reaching 4096 simultaneous queries in Infinity-8B.

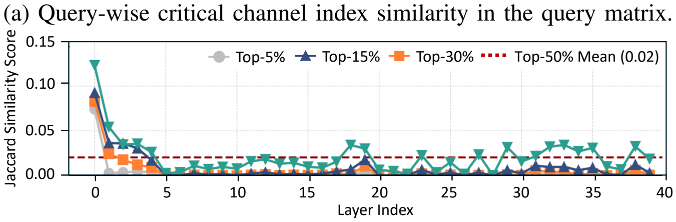
To quantify this effect, we analyze the per-stage latency breakdowns of Infinity-2B and 8B with InfiniGen, which uses channel reduction to reduce estimation latency, as shown in Fig. 3. Despite applying several methods to mitigate the latency of the criticality estimation stage, this stage still accounts for more than 71% of the end-to-end execution time at the final stage in both models. This large estimation latency lies directly on the critical path, substantially increasing the total execution time. As shown later, other approaches such as token pooling also fail to fully eliminate the criticality estimation overhead (see Fig. 10).

B. Accuracy Challenges of KV Retrieval in VAR

1) Row-wise channel distribution divergence: Beyond the latency overhead, VAR's multi-query generation also introduces a fundamental accuracy challenge for channel reduction. In LLM decoding, a reduced channel subset can be easily selected for a single query vector. In VAR, however, the



(a) Query-wise critical channel index similarity in the query matrix.



(b) Query-wise critical token index similarity in the score matrix.

Fig. 4: Evaluation of the Jaccard similarity between (a) critical channel indices identified from multiple query rows in the query matrix and (b) critical token indices identified from multiple query rows in the score matrix, across layers under various top-k settings.

important high-magnitude channels can vary across rows of the query matrix, as illustrated in Fig. 2b ①. This row-wise channel-importance divergence makes it difficult to identify a single reduced channel subset that accurately serves all queries. Thus, channels that are critical to certain query rows may be omitted from criticality estimation. As these omitted channels with large magnitude substantially contribute to the corresponding criticality scores, their exclusion may distort the relative ranking of KV entries and degrade retrieval accuracy.

To quantify this channel divergence, we measure the similarity between the important-channel sets selected by different query rows using the Jaccard score [6], $\mathrm{Jaccard}(A, B) = \frac{|A \cap B|}{|A \cup B|}$, which is widely used to measure the index similarity between two sets. The Jaccard score can evaluate whether different query rows select similar channel subsets or not. For all layers in the later stages (i.e., 8^{th} to 12^{th} stages), we vary the degree of channel reduction by selecting the top-30%, 15%, and 5% channels for each query row, and measure the similarity of the resulting channel index sets as shown in Fig. 4a. In Fig. 4a, the Jaccard score remains substantially low, with an average of 0.49 across layers and staying below 0.26 in early layers. This limited overlap confirms that important-channel sets differ substantially across query rows. Thus, selecting a single channel subset shared across all queries can omit channels that are critical to individual rows, leading to accuracy degradation.

2) Row-wise token distribution divergence: The divergence observed in the channel dimension also appears along the token dimension. In LLM decoding, token selection operates on a single score vector, as depicted in Fig. 2a ②. In contrast, VAR produces a score matrix with multiple query rows, where each row can exhibit a markedly different score distribution, as shown in Fig. 2b ②. As KV cache retrieval requires

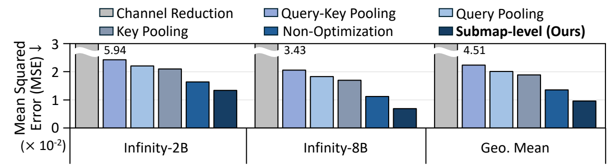


Fig. 5: MSE of the attention output matrix under different criticality estimation mechanisms, at the same 35.75% retrieval budget, measured against full-KV attention. Non-optimization computes criticality estimation with exact $\mathbf{QK}^\top$ and retrieves the top-35.75% tokens, whereas the reference preserves all KV tokens. Channel reduction retains 30% of the channels, and the pooling size is 32. Lower values indicate higher similarity.

materializing a single subset of key tokens shared across all queries, and fetching it to the GPU before attention execution, this row-wise token divergence makes retrieval challenging. Different query rows can exhibit different key preferences, making it difficult to determine a single subset.

To quantify this behavior, we measure the Jaccard similarity of the key index sets selected from different query rows in the score matrix under various top-k settings, similar to Fig. 4a. By varying the retrieval budget (i.e., the number of KV entries selected for retrieval), we evaluate whether different query rows consistently prefer similar key subsets. As shown in Fig. 4b, the key index sets selected by different query rows exhibit substantially low Jaccard similarity, with an average of 0.02 across layers. This low similarity indicates that selecting a single key subset across all query rows, which is required in KV retrieval, is challenging and can degrade the accuracy.

To further evaluate how the low similarity caused by diversity affects attention accuracy, we apply each method to reduce criticality estimation overhead and measure the mean squared error (MSE) of the resulting attention outputs, as shown in Fig. 5. Evaluated schemes include channel reduction to assess the impact of channel-importance diversity, and key pooling to assess key-token diversity. We also add query and query-key pooling to examine the effect of aggregating queries.

These schemes exhibit substantially higher MSE than Non-optimization (the design without optimization for reducing criticality estimation overhead), confirming that the row-wise diversity of channel and token importance limits the ability of prior optimization methods to preserve accurate attention outputs in VAR. In particular, query-side pooling is unsuitable for VAR because it merges query rows without considering their distinct key preferences. The pooled representation therefore fails to preserve the preference of each individual query, causing important keys for some queries to be omitted.

IV. Submap-Level Retrieval

As discussed in Section III, applying existing KV retrievals to VAR is not sufficiently effective. This motivates us to rethink the retrieval granularity itself. We therefore turn our attention to a granularity tailored to VAR, which we refer to as a *scale-submap (i.e., submap)*, and analyze the design space of submap-level KV retrieval. A *submap* is the contiguous chunk

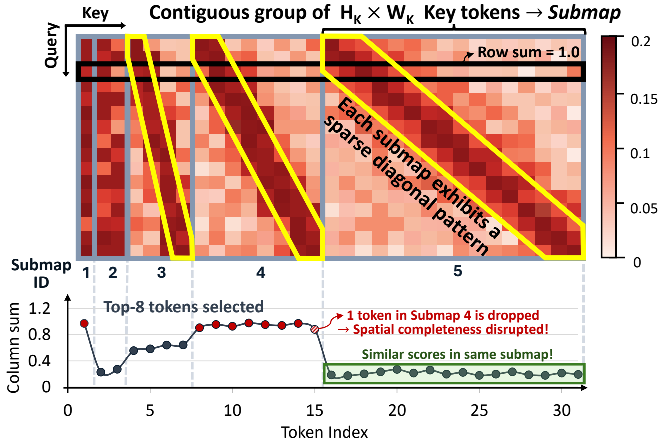


Fig. 6: Heat-map visualization of the probability matrix (P) of the 23rd head in the first layer of Infinity-8B. Submap refers to the contiguous group of tokens within the grey boundary.

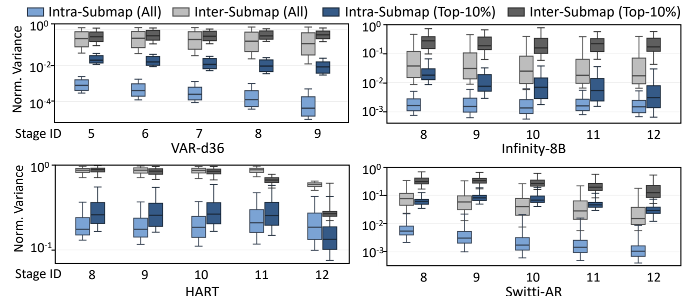


Fig. 7: Token-wise score variance across stages (y-axis in log scale) for intra-/inter-submap in VAR-d36, Infinity-8B, HART, and Switti-AR, in all heads and top-10% heads with the weakest intra-submap coherence in their aggregated key scores. The center line in each box denotes the mean, the box spans the 30%–70% percentiles, and the whiskers span the 20%–80% percentiles.

of $H_k \times W_k$ key and value tokens produced together at each resolution scale $\mathbf{r}_k \in [V]^{H_k \times W_k}$ of VAR. As all tokens at each resolution jointly form a single complete token map, this unit represents a full image at that specific resolution. Visually, each submap corresponds to the entire KV tokens separated by the grey boundary lines in Fig. 6.

A. Submap Coherence in VAR

As each submap corresponds to a group of KV tokens generated together at a distinct scale in VAR's next-resolution decoding process, we hypothesize that KV retrieval at the submap granularity may better align with the underlying attention structure than finer-grained retrieval methods. To examine this, we first analyze the attention patterns. In Fig. 6, the attention probability matrix exhibits a sparse multi-diagonal pattern consistent with the observations in prior work [32], [43], in which attention mass is concentrated along multiple narrow diagonal bands. Our analysis in Section III-B2 reveals that row-wise token divergence (Fig. 2b ②) makes this pattern challenging for prior token-level methods [32], [43], since a single shared subset cannot reliably preserve the distinct key preferences of all queries. In contrast, we turn our attention to the alignment between the multi-diagonal pattern and the scale-wise submap structure. This alignment indicates that VAR's attention structure and its token-wise divergence are organized at the submap level. We therefore consider this same pattern not as a challenge but as an opportunity to exploit VAR's submap-aligned structure.

To assess this opportunity, we analyze the column-wise sum, which serves as an aggregated measure of key importance, as shown at the bottom of Fig. 6. As high-score regions shift diagonally, aggregating the scores across queries distributes their high-score contributions evenly across the key tokens within each submap. Thus, key tokens within the same submap exhibit similar aggregated importance, whereas importance differs relatively more across submaps. This intra-submap coherence implies that tokens within the same submap may be considered collectively, providing a basis for criticality estimation and retrieval at the submap granularity.

To quantify this submap coherence, we analyze the intra- and inter-submap variance of key token importance across decoding stages, as shown in Fig. 7. Across all evaluated models and stages, normalized intra-submap variance remains consistently lower than inter-submap variance, with an average difference of 80.57%. This trend persists even among the 10% of attention heads exhibiting the weakest intra-submap coherence, identified by the largest variance ratio, $R_h = \frac{\text{var.}^{\text{intra}}(h)}{\text{var.}^{\text{inter}}(h)}$. Even for these heads, intra-submap variance remains 63.96% lower than inter-submap variance on average across models. These results indicate that submap coherence is a structural characteristic of VAR's next-resolution decoding rather than a pattern confined to a small subset of attention heads.

This coherence property can be understood from VAR's scale-wise decoding process. At each scale, VAR jointly generates $H_k \times W_k$ tokens to form a complete spatial token map. Each token therefore corresponds to a specific location in the underlying 2D grid, and together, these tokens preserve the spatial organization of the image at that scale. A submap is therefore not merely a contiguous group of keys in the KV cache, but a spatially organized structural unit, as illustrated in the left green box of Fig. 8. Submap-level retrieval preserves these spatial relationships, whereas finer-grained retrieval may retain only a fragmented portion of the token map, as illustrated in the left red box of Fig. 8. For example, as illustrated at the bottom of Fig. 6, top-k token-level retrieval with k=8 selects seven tokens from the 4^{th} submap and one from the 1^{st} submap. Thus, one token from the 4^{th} submap is omitted, preventing the corresponding submap from being preserved as a complete spatial token map.

Fig. 5 provides further evidence for the importance of preserving the spatial completeness of each submap. Submap-level selection (Ours) achieves lower attention-output error than exact token-level selection (Non-Optimization), which

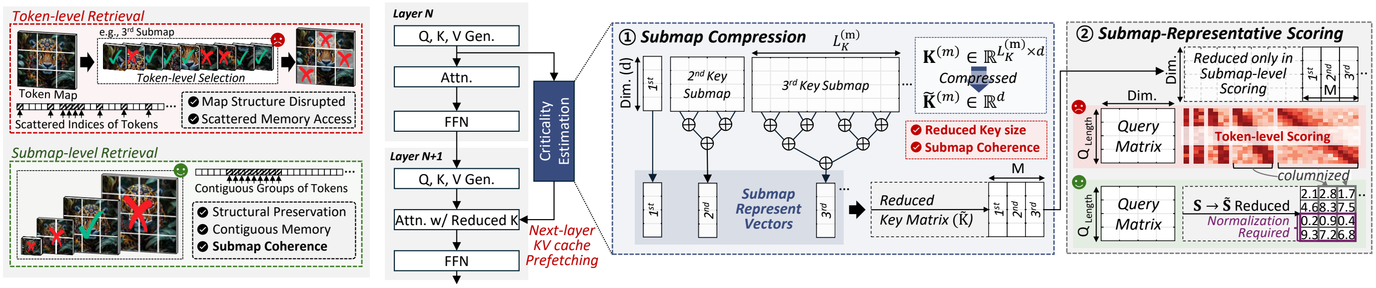


Fig. 8: Comparison of token-level and submap-level retrieval (left) and overview of the S2MR algorithm (right).

directly retrieves the tokens with the highest aggregated attention scores. This result indicates that preserving the complete spatial structure of a submap by submap-level selection can be more important than selecting individual high-importance tokens, motivating a retrieval design that treats each submap as a single selection unit.

B. S2MR Algorithm

Building on the demonstrated benefits of submap-level selection, we propose *S2MR* (Scale-Submap Granularity Retrieval), an efficient KV retrieval algorithm. As all tokens in a submap are selected together to preserve the spatial structure and completeness of the corresponding token map, criticality estimation only needs to determine which submaps should be fetched. A straightforward approach would be to construct the full score matrix using a large GEMM, aggregate the token-level scores within each submap, and rank the submaps based on their aggregated scores. Although this approach performs retrieval at the desired granularity, it remains inefficient as it explicitly computes scores for every key token, even though only one importance score is ultimately needed for each submap. To address this inefficiency, S2MR reformulates criticality estimation from token-level scoring to direct submap-level scoring by using representative vectors that preserve the aggregated score of each submap.

In detail, S2MR first compresses each key submap before criticality estimation (i.e., submap compression). Let the m^{th} key submap be represented as $\mathbf{K}^{(m)} \in \mathbb{R}^{L_K^{(m)} \times d}$, where $L_K^{(m)}$ is the number of tokens in the m^{th} submap and d is the hidden dimension. S2MR aggregates these tokens into a single representative vector as $\widetilde{\mathbf{K}}^{(m)} = \sum_{\ell=1}^{L_K^{(m)}} \mathbf{K}_\ell^{(m)} \in \mathbb{R}^d$, as Fig. 8①. M representative vectors are then concatenated within each attention head to form $\widetilde{\mathbf{K}} \in \mathbb{R}^{M \times d}$, where M denotes the number of submaps. Then, submap-representative scoring is performed as $\mathbf{Q}\widetilde{\mathbf{K}}^\top$ as Fig. 8②, reducing the scoring GEMM from $(L_Q, d) \times (d, L_K)$ to $(L_Q, d) \times (d, M)$. This reduces the effective key token length from L_K to M, lowering the complexity by a factor of L_K/M (e.g., 809× reduction in the final stage where $L_K = 10521$ and $M = 13$).

Despite substantially reducing the scoring complexity, representative scoring accurately reproduces the submap-wise aggregation of the full token-level scores. The score column for the m^{th} submap is computed as $\mathbf{Q}(\widetilde{\mathbf{K}}^{(m)})^\top$, which satisfies $\mathbf{Q}\big(\sum_{\ell=1}^{L_K^{(m)}} \mathbf{K}_\ell^{(m)}\big)^\top = \sum_{\ell=1}^{L_K^{(m)}} \mathbf{Q}(\mathbf{K}_\ell^{(m)})^\top$. Thus, each generated submap score vector is numerically equivalent to the vector obtained by first computing the full token-level score matrix with $\mathbf{Q}(\mathbf{K}^{(m)})^\top$ and summing entries into a vector.

As a result, representative scoring directly produces a compact score matrix $\widetilde{\mathbf{S}} \in \mathbb{R}^{L_Q \times M}$, instead of the large token-level score matrix $\mathbf{S} \in \mathbb{R}^{L_Q \times L_K}$. Each of the M columns ($\widetilde{\mathbf{S}}^{(m)}$) of the compact score matrix $\widetilde{\mathbf{S}}$ is the compressed score vector of each submap. S2MR then normalizes each query row to account for variations in score magnitude across query rows, and aggregates the normalized scores across query rows to obtain a single score per submap as $\widetilde{\mathbf{S}}_{\text{agg}} \in \mathbb{R}^M$ (the detailed execution flow is introduced in Section V-C). The submaps are then ranked according to $\widetilde{\mathbf{S}}_{\text{agg}}$, and the highest-ranked submaps are selected for KV retrieval to the GPU.

This reformulation reduces criticality estimation overhead by replacing token-level scoring with compact submap-level scoring, while maintaining accurate submap ranking and preserving complete spatial token maps during KV retrieval.

C. Benefits of S2MR

Computational efficiency. Although finer-grained methods reduce the effective key-token length through pooling or clustering, their groups are typically much smaller (e.g., 16/32/64 tokens) than a complete submap. Thus, the reduced key-side dimension remains relatively large, retaining substantial computational complexity and failing to effectively eliminate criticality estimation overhead. In contrast, since S2MR reduces the key-side dimension far more aggressively by compressing the entire submap (e.g., 4096 tokens in the last scale) into a single representative vector, it effectively reduces the scoring complexity and criticality estimation overhead.

Regular memory access pattern. Moreover, finer-grained methods often incur irregular memory access patterns and require sparse gathering from non-contiguous memory locations, introducing extra overhead during KV cache access. In contrast, since each submap is stored as a contiguous KV cache chunk, retrieval can be performed over contiguous memory regions, avoiding irregular access and sparse gathering overhead.

Retrieval accuracy. Simply increasing the group size in prior methods can reduce computation and irregular memory access, but introduces a fundamental limitation. Because groups are formed without leveraging submap boundaries, a single group

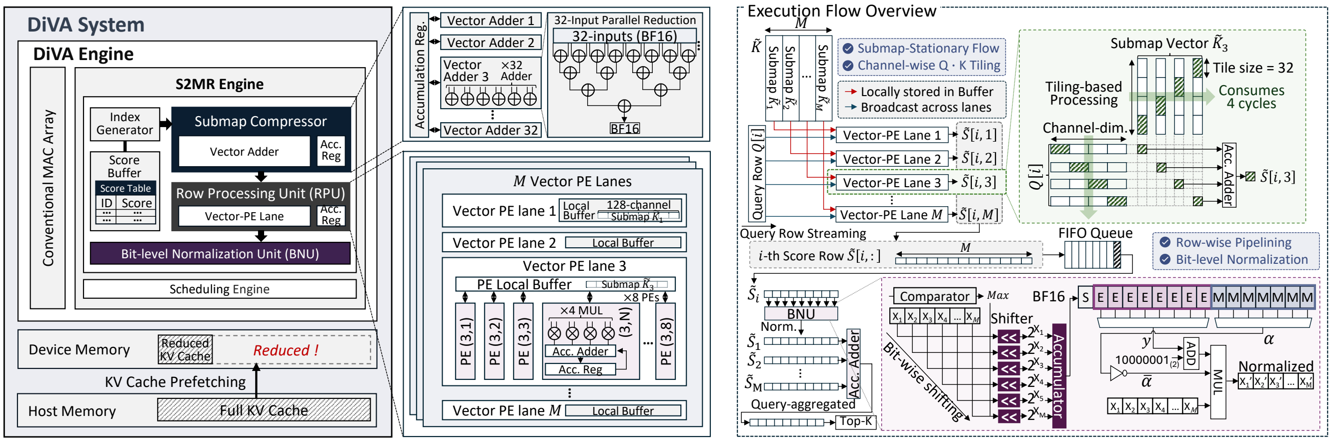


Fig. 9: Overview of the DiVA system architecture (left) and its execution flow (right). The S2MR Engine is integrated within DiVA and consists of the Submap Compressor, Row Processing Unit, Bit-level Normalization Unit, and Scheduling Engine.

may contain tokens from multiple submaps, though token importance can substantially vary across submaps. Therefore, these group-wise selection methods may either fetch low-priority tokens, reducing retrieval efficiency, or discard important tokens, degrading generation quality. Fig. 5 shows this, where submap-level selection (i.e., S2MR) achieves lower attention-output error than key pooling, highlighting the benefit of leveraging the submap structure.

Overall, by aligning retrieval with VAR's inherent submap structure, S2MR avoids the conventional granularity trade-off, where finer units improve selection precision but retain high scoring complexity, whereas coarser units reduce complexity at the expense of accuracy. S2MR greatly reduces scoring complexity without sacrificing accuracy and also enables regular memory access.

V. DiVA System Architecture

A. Opportunities of DiVA Hardware System

Although S2MR theoretically reduces the cost of submap-level criticality estimation by greatly reducing the scoring dimension, its benefits may not be fully realized in general-purpose accelerators. To reduce the effective key-side token length, S2MR performs submap compression, which aggregates all token vectors within each key submap. This large-scale vector reduction maps poorly onto general-purpose MAC arrays, which are optimized for dense matrix multiplication. Thus, submap compression accounts for more than 53% of the total S2MR execution time despite contributing only 4.3% of its FLOPs, revealing an opportunity to consider specialized hardware that accelerates submap aggregation.

Beyond submap compression, S2MR transforms scoring into an asymmetric $(L_Q, d) \times (d, M)$ multiplication, where M is far smaller than L_Q (L_Q is 315× larger than M in later scales). When the small output dimension M falls below the tile width of conventional dense-matrix engines, it causes partial tile utilization or zero-padding, wasting compute cycles and degrading energy efficiency. The workload also exhibits an imbalance in operand-reuse patterns. The small set of submap vectors is reused across all L_Q query rows, whereas each query row is reused across only M vectors. Although general-purpose dataflows can exploit operand reuse, they are not specifically organized around this extreme reuse imbalance. These characteristics motivate the design of a specialized scoring engine and dataflow that exploit the small output dimension and asymmetric operand reuse.

Finally, since score magnitudes vary substantially across the rows of the score matrix $\widetilde{\mathbf{S}} \in \mathbb{R}^{L_Q \times M}$ as shown in Fig. 8, row-wise normalization is required to prevent high-magnitude rows from dominating the aggregated submap scores. Unlike conventional accelerators that provide dedicated support for exponential computation (i.e., Softmax), S2MR does not require exact Softmax probabilities during criticality estimation. Instead, it only needs to preserve the coarse-grained priority ordering among submaps, allowing normalization to be implemented using substantially cheaper operations without costly exponential units. Moreover, since each score row contains only M elements, all elements can be processed fully in parallel with minimal area overhead. Together, these properties expose an opportunity to design a lightweight normalization unit tailored to S2MR.

B. DiVA Architecture Overview

To realize these opportunities, we propose DiVA, a system designed to support S2MR-based KV retrieval. Fig. 9 presents its overall architecture, which consists of a CPU with high-capacity host memory, low-capacity device memory, and the DiVA Engine. The full KV cache is offloaded to host memory, while device memory temporarily holds the fetched KV submaps selected after criticality estimation. DiVA integrates a conventional MAC array for the primary non-retrieval computations (e.g., self-attention, FFN layers), while offloading KV retrieval computations to the S2MR Engine, which consists of a Submap Compressor, Row Processing Unit, Bit-level Normalization Unit, and Scheduling Engine.

C. Detailed Hardware Design and Execution Flow

MAC Array. The MAC array performs Transformer computations (linear projection, attention, FFN) outside the retrieval path, which decouples the retrieval process and enables DiVA to specialize the retrieval path for submap prioritization.

Index Generator and Score Table. Index Generator fills the table entry with each submap index as (batch ID, head ID, submap ID) in the Score Table within the Score Buffer. After criticality estimation, the representative score of each submap is written to its corresponding index entry.

Submap Compressor. The Submap Compressor aggregates all tokens within each key submap into a single representative vector $\tilde{K}^{(m)}$. The compressor consists of a BF16 adder array organized as adder-tree structures with accumulation registers. It contains 32 vector adders, allowing it to process 32 key-token vectors across 32 channel elements in parallel during each reduction cycle. Accumulation registers store intermediate partial sums and carry them forward across successive reduction cycles until the final vector is produced. As each token in a submap is produced once and does not change afterward, its representative vector can be computed once and reused, rather than being recomputed on every step. As key tokens of a submap are generated by the MAC array after linear projection and written back to device memory, the compressor concurrently aggregates them into a representative vector, fusing compression into the write-back path and eliminating redundant aggregation and full KV cache access across steps.

Row Processing Unit (RPU). The RPU computes submap-representative scoring with the compressed submap vectors and forwards each completed score row to the subsequent stage. As introduced in Section V-A, the resulting score computation $(L_Q, d) \times (d, M)$ has a highly asymmetric shape and imbalance in its operand-reuse pattern. To exploit these, RPU adopts a submap-stationary dataflow that stores the highly reused M submap vectors in PE-local buffers while streaming query rows. As each submap vector occupies only 256 B in BF16, all M vectors can be stored in their respective local buffers throughout the processing of the entire query stream within each attention head. This maximizes local data reuse and eliminates repeated memory accesses to load submap vectors. Within this flow, each lane computes the dot product between one streamed query row and one stationary submap vector. In detail, RPU assigns each of the M submap vectors to a dedicated vector processing element (Vector-PE) lane. Each lane contains 32 MAC units and processes the 128-channel dimension over four 32-channel tiles. For each streamed query row, the same query tile is broadcast to all M lanes, while each lane multiplies it by the corresponding tile of its assigned submap vector and accumulates the partial sums locally in the accumulation register. After 4 cycles, the lanes independently produce the M submap scores, which are directly assembled into the complete score row $\tilde{S}[i, 1:M]$, without requiring inter-lane data movement. As query rows are continuously streamed via RPU, the stationary submap vectors remain in the local buffers and are repeatedly reused across all passes. RPU generates a complete score row and immediately forwards it to the subsequent stage without waiting for the remaining rows, forming a tightly coupled row-streaming pipeline.

TABLE I: Hardware configuration of DiVA (e.g., $M = 13$)

MAC Array	32-lane BF16 MAC Unit × 1344 per MAC Unit: 32 Mul, 32 Add, 32 Acc. Reg.
Submap Compressor	32 vector adders (32-ch BF16) per Vector Adder: 32-input Adder Tree 32 Acc. Reg. (i.e., Accumulation Register)
RPU	M Vector-PE lanes (32-wide) per lane: 8 PEs (w/ 4 MUL, 4 ADD, 1 Acc. Reg.)
BNU	M Shifter / M Sub / M Mul / $M-1$ Adder $M-1$ Comparator / 1 Inverter

Bit-level Normalization Unit (BNU). Each score row produced by RPU must be normalized before query-wise aggregation, as score magnitudes vary across query rows (Fig. 8), which causes high-magnitude rows to dominate the aggregated score, leading to inaccurate submap selection. Simple linear normalization methods, such as max- or sum-based normalization, are inexpensive but ineffective [48], as they lack the amplification effect needed to sharpen score differences and emphasize dominant submaps. Also, exact Softmax is unnecessarily expensive since criticality estimation is performed on a separate retrieval path from attention (Fig. 8), and therefore S2MR requires only reliable submap ranking for retrieval, rather than an exact attention probability distribution. Building on Softermax [48], the Bit-level Normalization Unit (BNU) provides the required score amplification at low hardware cost by replacing base-e exponentiation with base-2 shifting: $p_i = \frac{2^{s_i - \max(\mathbf{s})}}{\sum_{j=1}^{M} 2^{s_j - \max(\mathbf{s})}}$. As the row length is reduced to M, this compact row dimension keeps the number of required shifters small (e.g., 13), allowing BNU to fully parallelize bit shifting operations across all row elements with minimal area overhead. To further simplify division, BF16 denominator is represented as $(1+\alpha)2^y$, yielding $\frac{2^{s_i - \max(\mathbf{s})}}{(1+\alpha)2^y} \approx (1-\alpha)2^{s_i - \max(\mathbf{s}) - y}$. The term $(1-\alpha)$ is obtained with a bit inverter, eliminating a general-purpose divider. Although BNU normalizes each row only by bit-level operations, it closely preserves Softmax accuracy, yielding nearly identical FID scores (lower is better) on both Infinity-2B (3.11 vs. 3.09) and Infinity-8B (2.24 vs. 2.21) in our evaluation. Each normalized row is immediately accumulated into a running M-element score vector, avoiding materialization of the full $\tilde{\mathbf{S}} \in \mathbb{R}^{L_Q \times M}$ matrix. After all query rows are processed, the submaps with the highest scores are selected for retrieval.

Scheduling Engine. The Scheduling Engine orchestrates the DiVA system and uses 2 optimizations to improve end-to-end performance. First, it schedules next-layer KV prefetching to overlap PCIe transfers with GPU computation, enabling intra-stage latency hiding, as in conventional retrieval systems. Second, it organizes the fine-grained execution of a 3-stage row-wise streaming pipeline, which consists of row-wise scoring, normalization, and aggregation, so that the stages are tightly overlapped. This row-streaming organization is critical for latency reduction as (1) it eliminates the data movement

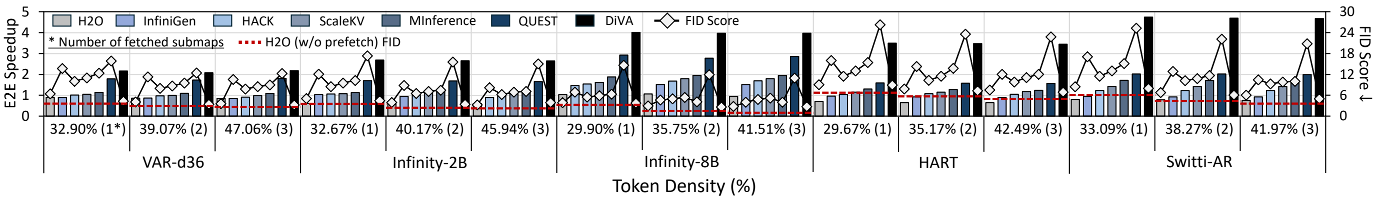


Fig. 10: Comparison of speedup and accuracy across various density settings. Speedups are normalized to Full KV Fetch.

of the entire intermediate score matrix and (2) allows scoring and normalization to proceed in a pipelined manner, which avoids the stall inefficiency arising from inter-row processing time imbalance.

VI. Evaluations

A. Experimental Setup

Evaluation Models and Datasets. We utilize 3 VAR models: VAR-d36 [56], Infinity-2B, Infinity-8B [15], and 2 next-resolution prediction models: Switti-AR [58] and HART [53]. For accuracy evaluation, we adopt FID (Fréchet Inception Distance [17]), where lower values indicate better performance. For VAR-d36, we use ImageNet [9], following its class-conditioned image generation. For Infinity-2B/-8B, HART, and Switti-AR, we use the GenEval dataset [13] to perform text-conditioned image generation. Images generated by Full KV Fetch are used as the reference dataset for FID, as our goal is to measure how well each method preserves original model accuracies while improving performance.

Baseline Implementations. We consider 7 scenarios: Full KV Fetch, H2O [68], InfiniGen [30], MInference [20], QUEST [54], HACK [43], and ScaleKV [32]. Full KV Fetch does not perform criticality estimation and loads the processed layer's full KV cache from CPU memory for attention, preserving the original model accuracy. H2O fetches only the most critical tokens by exact $QK^\top$ criticality estimation and performs token-wise top-k selection, without any optimizations (e.g., channel reduction or token pooling). InfiniGen only keeps the top-30% of largest-magnitude channels, following the original setting. MInference applies average block pooling with a size of 32, reducing both query and key token lengths by 32× before multiplication. QUEST utilizes 16-token pages [28], reduces the query matrix to a single vector by summing across the token length dimension, and performs $Q \cdot K$-based scoring and fetching at the page level, following its original design. HACK uniformly samples 32 queries, while ScaleKV selects 16 queries using 2D-spatial patch centroids. Both perform criticality estimation by using only these queries while scoring the full KV cache, and retrieve KV entries at the token level. We adapt H2O, MInference, QUEST, HACK, and ScaleKV to the KV retrieval setting by using their criticality estimation mechanisms to selectively fetch KV entries from CPU memory. We enable KV offloading from stage 6 for VAR-d36 and stage 8 for Infinity, HART, and Switti-AR, where KV cache growth begins to pressure GPU DRAM capacity.

DiVA Hardware Implementation. We developed an in-house cycle-level simulator and integrated DRAMSim3 [34]. We model an NVIDIA RTX A6000 GPU with 48 GB of device memory and a DDR4-based host memory of the AMD EPYC™ 7543 processor, and integrate them into our simulator. For PCIe data transfer, we measure the effective bandwidth, latency, and utilization in a real server environment where the processors above are connected by PCIe 4.0 with 32 GB/s bandwidth, and feed these measured values into our simulator. For a fair comparison against baselines, we configure the MAC array of the DiVA system to match the throughput of the Tensor Cores in the NVIDIA RTX A6000 GPU (154.8 TFLOPS). All components of DiVA are designed to operate at the same clock frequency as the A6000 GPU (1.8 GHz). Detailed configurations of DiVA are shown in Table I. We implement the DiVA core units using Synopsys Design Compiler and synthesize them with the FreePDK 45 nm library [49], while the memory modules are modeled with CACTI [3]. GPU power is measured by NVIDIA-SMI, and PCIe utilization is measured by NVIDIA Nsight Systems [41].

B. Performance and Efficiency Analysis

1) Speedup and Accuracy: We compare the end-to-end speedup and FID score under various retrieval density settings, as shown in Fig. 10. For a fair comparison, we first measure the retrieval density of DiVA for each head and then apply the same density to all baselines. Across all density settings, DiVA achieves the highest speedup. DiVA achieves a 3.26× speedup over Full KV Fetch, and provides speedups of 4.23×, 3.22×, 2.83×, 2.60×, 2.32×, and 1.71× over H2O, InfiniGen, HACK, ScaleKV, MInference, and QUEST, respectively. H2O and InfiniGen are slower than Full KV Fetch as their token-level criticality estimation incurs great overhead, which exceeds the performance benefits achieved by selective retrieval. HACK and ScaleKV only reduce the query-side token length and include the entire key tokens during criticality estimation. Therefore, the estimation cost continues to scale with the rapidly growing key length, limiting their speedup. DiVA reduces criticality estimation latency via its co-designed algorithm and hardware, allowing it to more effectively remove the estimation overhead and achieve the highest end-to-end performance.

DiVA also achieves the best accuracy among baselines. HACK and ScaleKV estimate token importance using only a subset of query rows, which may fail to capture the distinct key preferences of omitted queries. Their token-level selection also may fragment the submap structure and degrade accuracy.

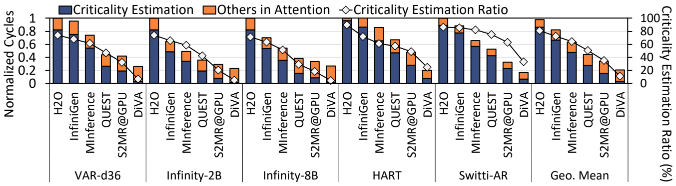


Fig. 11: Execution cycle breakdown and criticality estimation ratio (%) in the attention layer. Cycle breakdown is normalized to H2O.

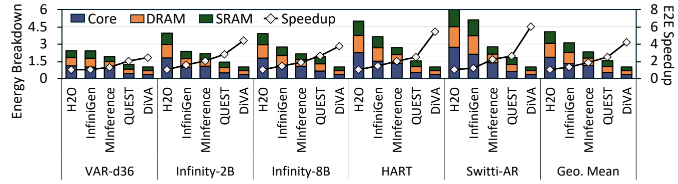
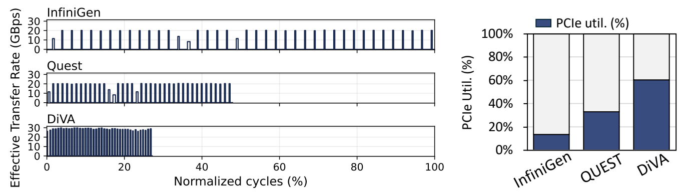


Fig. 12: Comparison of energy consumption breakdown and speedup. Energy breakdown is normalized to DiVA, and speedup is normalized to H2O.



(a) Effective PCIe data transfer rate.

(b) PCIe utilization.

Fig. 13: Comparison of effective PCIe data transfer rate and PCIe utilization across InfiniGen, QUEST, and DiVA.

DiVA matches or even surpasses H2O at the same retrieval density, although H2O selects the individual important tokens most accurately under the same next-layer prefetching setting. DiVA often achieves comparable accuracy to H2O while fetching fewer submaps. This indicates that, for a target accuracy level, DiVA can use a lower retrieval density and therefore achieve higher speedup. DiVA exhibits comparable accuracy to H2O (w/o prefetch), which performs exact token-level estimation without any approximation or next-layer prediction and therefore can be regarded as a strong KV retrieval accuracy baseline. These indicate that, for KV retrieval in VAR models, preserving the structural completeness within each submap is more effective than selecting individual tokens based on fine-grained importance, as simultaneously generated tokens in each submap are spatially adjacent and contiguously form the total image structure. This differs from LLMs, where tokens are discrete and token-level importance estimation is effective, showing that DiVA's submap-level strategy is efficient and accurate.

2) Latency Breakdown: As DiVA mainly targets criticality estimation in KV retrieval, we further analyze the latency breakdown of the attention layer, including the criticality estimation stage, as shown in Fig. 11. Across baselines, criticality estimation dominates execution, accounting for 87.54%, 80.81%, 70.43%, and 56.74% of the attention layer cycles in H2O, InfiniGen, MInference, and QUEST, respectively. The baselines insufficiently address the estimation cost caused by VAR's multi-query decoding, leaving it as the primary bottleneck that inflates attention layer latency. S2MR@GPU, which denotes executing S2MR on an A6000 GPU, reduces this fraction to 35.34%, showing that the algorithm alone alleviates a substantial portion of the estimation overhead. DiVA further reduces it to only 14.74%, showing that DiVA greatly reduces the estimation overhead, and thus criticality estimation is no longer the bottleneck. This result implies that although S2MR greatly reduces criticality estimation overhead, its benefits are not fully realized in GPUs. DiVA addresses the remaining inefficiencies via specialized hardware and execution flow, further improving performance over S2MR@GPU and showing that effectively reducing the bottleneck requires not only algorithmic advances but also co-designed hardware support.

3) Energy Efficiency: As shown in Fig. 12, we also compare the energy consumption breakdown of the DiVA system against baselines. DiVA improves energy efficiency by 3.02×, 2.35×, 2.03×, and 1.66× over H2O, InfiniGen, MInference, and QUEST. This improvement comes from reductions in both computation and memory access energy. DiVA reduces core unit energy by reducing the execution cycles of the criticality estimation stage. Compared with H2O, this reduces core energy by 81.24%. Second, submap-granularity retrieval allows DiVA to access contiguous KV cache chunks (i.e., submaps) instead of irregularly scattered token-level entries. This improves DRAM access efficiency and reduces DRAM energy by 71.87%. Finally, the submap-wise access pattern also improves SRAM locality, reducing SRAM energy by 66.24% compared with H2O.

4) PCIe Utilization: We compare the effective data transfer rate and PCIe utilization ratio. Fig. 13a presents the effective PCIe data transfer rate, computed as transferred data size divided by transfer latency. DiVA reduces PCIe idle time by lowering criticality estimation latency. Submap-level fetching enables larger contiguous fetches, improving the effective transfer rate, resulting in a denser transfer pattern over cycles than InfiniGen and QUEST. Fig. 13b shows the PCIe utilization ratio, computed as the fraction of PCIe-utilized cycles over total cycles. DiVA achieves 61.31%, higher than the 13.47% of InfiniGen and the 32.99% of QUEST. Results imply that DiVA uses PCIe bandwidth more efficiently than baselines, minimizing PCIe idle time and allowing PCIe transfers to overlap more effectively with other operations.

C. Area and Power Analysis

Table II reports the area and power breakdown of DiVA. S2MR engine, which consists of Submap Compressor, BNU, and RPU, occupies 2.06 mm^2, accounting for 2.76% of the total area. Its power consumption is 827.11 mW, corresponding to 1.63% of the total system power. Score Buffer stores submap scores using 1 byte per submap, and requires 3 KB of SRAM, which is negligible compared with the SRAM used by

TABLE II: Area and power breakdown of DiVA.

Hardware Components	Area [mm^2]	Area [%]	Power [mW]	Power [%]	Freq.
MAC Array	72.49	97.24	49831.43	98.37	1.8 GHz
Submap Compressor	1.84	2.47	768.77	1.52	
BNU	0.11	0.15	2.62	0.01	
Row Processing Unit	0.11	0.14	55.72	0.10	
Total	74.55	100	50658.54	100	

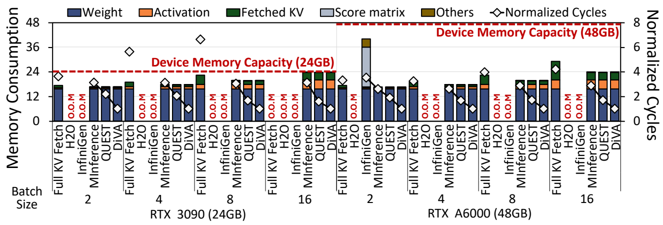


Fig. 14: Memory consumption breakdown and execution cycle comparison across batch sizes for Infinity-8B at a retrieval density of 35.75%. Execution cycles are normalized to DiVA in each batch size.

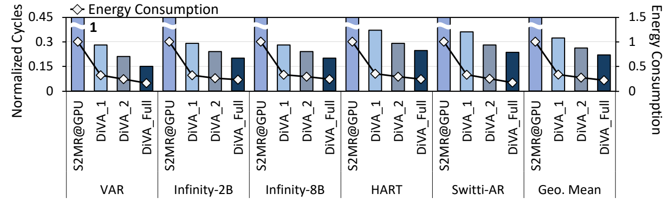


Fig. 15: Evaluation of each design component in terms of normalized cycles and energy consumption. Cycles and energy consumption are both normalized to S2MR@GPU.

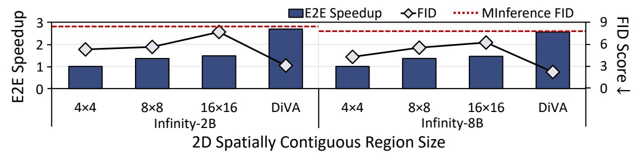


Fig. 16: Comparison of speedup and FID score across different spatial contiguous retrieval methods. Speedups are normalized to 4×4 spatial contiguous retrieval of each model.

the MAC array matched to the throughput of an A6000 GPU (over 10 MB). Results show that integrating the S2MR engine incurs reasonable area, power, and memory overhead.

D. Design Space Exploration

1) Batch Variation: We evaluate the memory consumption breakdown and latency under 24 and 48 GB (matching the memory capacity of RTX 3090 and A6000), as shown in Fig. 14. H2O and InfiniGen require materializing a large intermediate score matrix during criticality estimation, causing OOM errors under a 24 GB configuration even at batch size 2. At batch size 2 under 48 GB, both materialize the same score matrix, but InfiniGen's channel reduction keeps its peak memory consumption below the limit by reducing intermediate memory, while H2O does not. Among methods within given memory budgets, DiVA achieves the lowest latency while preserving the best accuracy, as discussed in Fig. 10.

2) Impact of Each Design Component: We evaluated how each design component of DiVA contributes by showing the cumulative benefit (Fig. 15) in the criticality estimation stage. DiVA_1 executes S2MR on the proposed Submap Compressor and RPU, with Softmax for normalization. DiVA_2 executes S2MR with DiVA hardware and processes normalization with BNU. DiVA_Full denotes the full DiVA system, including a row-wise streaming pipeline. DiVA_1 and DiVA_2 achieve 3.11× and 3.82× speedup compared to S2MR@GPU, respectively. DiVA_Full achieves the best overall performance, yielding an average 5.06× speedup and 4.61× energy-efficiency improvement compared to S2MR@GPU. This shows that DiVA successfully resolves the inefficiency in S2MR@GPU through its dedicated hardware and execution flow, fully leveraging the algorithmic benefits while incurring minimal area overhead, as shown in Table II.

3) Intra-submap Granularity Sweep: Since VAR generates images through 2D token maps, we further examine whether preserving spatial locality in the token-map layout may also be beneficial. To examine this possibility, we reshape the 1D-flattened KV cache into its 2D $H_k \times W_k$ token-map layout and use 2D spatially contiguous regions (4×4, 8×8, 16×16) in the KV cache as retrieval units, all under the same per-head density as shown in Fig. 16. Notably, all spatially contiguous region retrieval methods outperform MInference (i.e., 1D token index-based pooling), confirming that preserving 2D spatial structure is beneficial. However, despite providing greater selection flexibility, finer spatial regions consistently yield worse FID than submap-level selection. This is because they still split each submap into finer-grained retrieval units, which may fragment the structural organization within each submap. This offsets the potential benefit of finer-grained precise selection. This result indicates that aligning retrieval units with VAR's native submap structure is more important for accuracy than increasing token-level selection precision. Moreover, spatially contiguous methods reduce the key dimension only in proportion to the region size, limiting the effect of token length reduction compared to DiVA (e.g., to $1/16$, $1/64$, and $1/256$ of a 4096-token submap). This leaves the scoring complexity much larger (i.e., $256\times$, $64\times$, and $16\times$ each) than DiVA, which reduces each submap to a single vector and thus limits the speedup (1.00/1.35/1.49× vs. 2.68× on Infinity-2B).

VII. Conclusion

DiVA is the first co-designed architecture for KV cache retrieval in VAR models, to our knowledge. S2MR shifts the retrieval granularity from tokens to submaps and effectively reduces the criticality estimation overhead. To efficiently support S2MR, DiVA incorporates the S2MR Engine for submap compression, row-wise processing, and lightweight normalization. DiVA achieves up to 4.23× end-to-end speedup and 3.02× energy-efficiency improvement while maintaining generation quality close to ideal token selection.

REFERENCES

[1] M. Adnan, A. Arunkumar, G. Jain, P. J. Nair, I. Soloveychik, and P. Kamath, "Keyformer: Kv cache reduction through key tokens selection for efficient generative inference," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 114–127, 2024.

[2] H. Ahn, R. Hwang, and M. Rhu, "Exploring kv cache quantization in multimodal large language model inference," *IEEE Computer Architecture Letters*, vol. 25, no. 1, pp. 13–16, 2025.

[3] R. Balasubramonian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "Cacti 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 14, no. 2, pp. 1–25, 2017.

[4] A. R. Bambhaniya, S.-C. Kao, G. Jeong, S. Subramanian, A. Yazdanbaksh, and T. Krishna, "Demystifying the cost versus benefits of sparse llm acceleration," *IEEE Micro*, 2026.

[5] J. Cederlund, A. Berg, W. Isaksson, D. A. E. Acar, C. Zhou, and P. Giselsson, "HeatKV: Head-tuned KV-cache compression for visual autoregressive modeling," 2026.

[6] S. Ceri, A. Bozzon, M. Brambilla, E. Della Valle, P. Fraternali, and S. Quarteroni, "An introduction to information retrieval," in *Web Information Retrieval*. Berlin, Heidelberg: Springer, 2013, pp. 3–11.

[7] D. Chen, J. Cong, A. Mirhoseini, C. Kozyrakis, S. Mitra, J. Xiong, C. Young, A. Anandkumar, M. Littman, A. Kirschen, S. Shao, S. Leef, N. Shanbhag, D. Milojicic, M. Schulte, G. Cauwenberghs, J. M. Chow, T. Dao, K. Gopalakrishnan, R. Ho, H. Kim, K. Olukotun, D. Z. Pan, M. Ren, D. Roth, A. Singh, Y. Sun, Y. Wang, Y. LeCun, and R. Puri, "Ai+ hw 2035: Shaping the next decade," *arXiv preprint arXiv:2603.05225*, 2026.

[8] W. Chen, S. He, H. Qu, R. Zhang, S. Yang, P. Chen, Y. Zheng, B. Huai, and G. Chen, "IMPRESS: An Importance-Informed Multi-Tier prefix KV storage system for large language model inference," in *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. Santa Clara, CA: USENIX Association, Feb. 2025, pp. 187–201.

[9] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," *2009 IEEE conference on computer vision and pattern recognition*, pp. 248–255, 2009.

[10] P. Dong, J. Zhuang, Z. Yang, S. Ji, Y. Li, D. Xu, H. Huang, J. Hu, A. K. Jones, Y. Shi, Y. Wang, and P. Zhou, "Eq-vit: Algorithm-hardware co-design for end-to-end acceleration of real-time vision transformer inference on versal acap architecture," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 11, pp. 3949–3960, 2024.

[11] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," *International Conference on Learning Representations*, 2021.

[12] R. Ghadia, A. Kumar, G. Jain, P. Nair, and P. Das, "Dialogue without limits: Constant-sized kv caches for extended responses in llms," *arXiv preprint arXiv:2503.00979*, 2025.

[13] D. Ghosh, H. Hajishirzi, and L. Schmidt, "Geneval: An object-focused framework for evaluating text-to-image alignment," *Advances in Neural Information Processing Systems*, vol. 36, pp. 52 132–52 152, 2023.

[14] Y. Gu, A. Khadem, S. Umesh, N. Liang, X. Servot, O. Mutlu, R. Iyer, and R. Das, "Pim is all you need: A cxl-enabled gpu-free system for large language model inference," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 862–881.

[15] J. Han, J. Liu, Y. Jiang, B. Yan, Y. Zhang, Z. Yuan, B. Peng, and X. Liu, "Infinity: Scaling bitwise autoregressive modeling for high-resolution image synthesis," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 15 733–15 744.

[16] J. Hao, Y. Zhu, T. Wang, J. Yu, X. Xin, B. Zheng, Z. Ren, and S. Guo, "OmniKV: Dynamic context selection for efficient long-context LLMs," in *The Thirteenth International Conference on Learning Representations*, 2025.

[17] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "Gans trained by a two time-scale update rule converge to a local nash equilibrium," *Advances in neural information processing systems*, vol. 30, 2017.

[18] C. Hooper, S. Kim, H. Mohammadzadeh, H. Genc, K. Keutzer, A. Gholami, and Y. Sophia Shao, "Speed: Speculative pipelined execution for efficient decoding," in *Enhancing LLM Performance: Efficacy, Fine-Tuning, and Inference Techniques*. Springer, 2025, pp. 19–32.

[19] N. Iliakopoulou, J. Stojkovic, C. Alverti, T. Xu, H. Franke, and J. Torrellas, "Chameleon: Adaptive caching and scheduling for many-adapter llm inference environments," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, 2025, pp. 217–231.

[20] H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, S. Ahn, Z. Han, A. H. Abdi, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, "Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention," *Advances in Neural Information Processing Systems*, vol. 37, pp. 52 481–52 515, 2024.

[21] D. Joo, H. Hosseini, R. Hadidi, and B. Asgari, "Mustafar: Promoting unstructured sparsity for kv cache pruning in llm inference," *arXiv preprint arXiv:2505.22913*, 2025.

[22] D. Kim, S. Yang, W. Shin, and J.-Y. Kim, "V-rex: Real-time streaming video llm acceleration via dynamic kv cache retrieval," *arXiv preprint arXiv:2512.12284*, 2025.

[23] J.-H. Kim, J. Kim, S. Kwon, J. W. Lee, S. Yun, and H. O. Song, "Kvzip: Query-agnostic kv cache compression with context reconstruction," *arXiv preprint arXiv:2505.23416*, 2025.

[24] Y. Kim, F. Tahmasebi, G. Koo, and H. Kwon, "Logflex: Flexible-bit log arithmetic accelerator for language models on edge," *IEEE Micro*, 2026.

[25] A. Kuzina, M. Pióro, and B. E. Bejnordi, "Kava: Latent reasoning via compressed kv-cache distillation," in *The Fourteenth International Conference on Learning Representations*, 2026.

[26] H. Kwon, K. Koo, J. Kim, W. Lee, M. Lee, G. Jung, H. Lee, Y. Jung, J. Park, Y. Song, B. Yang, H. Choi, G. Kim, J. Won, W. Shin, C. Kim, G. Shin, Y. Kwon, I. Kim, E. Lim, J. Kim, and J. Choi, "Pimphony: Overcoming bandwidth and capacity inefficiency in pim-based long-context llm inference system," in *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2026, pp. 1–21.

[27] H. Kwon, K. Koo, J. Kim, W. Lee, M. Lee, H. Lee, Y. Jung, J. Park, Y. Song, B. Yang, H. Choi, G. Kim, J. Won, W. Shin, C. Kim, G. Shin, Y. Kwon, I. Kim, E. Lim, J. Kim, and J. Choi, "Lol-pim: Long-context llm decoding with scalable dram-pim system," *arXiv preprint arXiv:2412.20166*, 2024.

[28] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *SOSP '23: Proceedings of the 29th Symposium on Operating Systems Principles*. ACM, 2023, pp. 611–626.

[29] S. Lee, H. Kim, S. Hwang, G. Heo, M. Noh, and J. Huh, "Throughput-oriented llm inference via kv-activation hybrid caching with a single gpu," in *2025 IEEE 43rd International Conference on Computer Design (ICCD)*. IEEE, 2025, pp. 269–276.

[30] W. Lee, J. Lee, J. Seo, and J. Sim, "Infinigen: Efficient generative inference of large language models with dynamic kv cache management," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 155–172.

[31] C. Li, Y. Yin, C. Xue, Z. Wang, F. Bai, Y. Guo, X. Jiang, Q. Wu, Y. Xie, and G. Sun, "Hardware-software co-design for 3d-dram-based llm serving accelerator," *arXiv preprint arXiv:2603.04797*, 2026.

[32] K. Li, Z. Chen, C.-Y. Yang, and J.-N. Hwang, "Memory-efficient visual autoregressive modeling with scale-aware kv cache compression," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[33] R. Li, S. Pal, V. N. Pullu, P. Sinha, J. Ryoo, L. K. John, and N. J. Yadwadkar, "Oneiros: Kv cache optimization through parameter remapping for multi-tenant llm serving," in *Proceedings of the 2025 ACM Symposium on Cloud Computing*, 2025, pp. 88–101.

[34] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, "Dramsim3: A cycle-accurate, thermal-capable dram simulator," *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.

[35] T. Li, D. Katabi, and K. He, "Self-conditioned image generation via generating representations," *arXiv preprint arXiv:2312.03701*, 2023.

[36] C. Lin, J. Tang, S. Yang, H. Wang, T. Tang, B. Tian, I. Stoica, S. Han, and M. Gao, "Twilight: Adaptive attention sparsity with hierarchical top-p pruning," in *Advances in Neural Information Processing Systems*, vol. 38, 2025.

[37] G. Liu, C. Li, Z. Ning, J. Lin, Y. Yao, D. Ke, M. Guo, and J. Zhao, "Freekv: Boosting kv cache retrieval for efficient llm inference," *arXiv preprint arXiv:2505.13109*, 2025. [Online]. Available: https://arxiv.org/abs/2505.13109

[38] S. Moon, J. Cha, H. Park, and J.-Y. Kim, “Hybe: Gpu-npu hybrid system for efficient llm inference with million-token context window,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 808–820.

[39] S. Na, G. Jeong, B. H. Ahn, A. Jezghani, J. Young, C. J. Hughes, T. Krishna, and H. Kim, “Flexinfer: Flexible llm inference with cpu computations,” *Proceedings of Machine Learning and Systems*, vol. 7, 2025.

[40] S. Na, G. Jeong, B. H. Ahn, J. Young, T. Krishna, and H. Kim, “Understanding performance implications of llm inference on cpus,” in *2024 IEEE international symposium on workload characterization (IISWC)*. IEEE, 2024, pp. 169–180.

[41] NVIDIA, “NVIDIA Nsight Systems,” 2026, accessed: 2026-03-27. [Online]. Available: https://developer.nvidia.com/nsight-systems

[42] W. Peebles and S. Xie, “Scalable diffusion models with transformers,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, pp. 4195–4205.

[43] Z. Qin, Y. Lv, M. Lin, H. Guo, Z. Zhang, D. Zou, and W. Lin, “Head-aware KV cache compression for efficient visual autoregressive modeling,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 30, pp. 24 982–24 990, 2026.

[44] D. Quinn, E. E. Yücel, J. Kim, J. F. Martínez, and M. Alian, “Longsight: Compute-enabled memory to accelerate large-context llms via sparse attention,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, 2025, pp. 34–48.

[45] A. Ramachandran, S. Kundu, A. Raha, S. Kundu, D. K. Mathaikutty, and T. Krishna, “Accelerating llm inference with flexible n: M sparsity via a fully digital compute-in-memory accelerator,” in *2025 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE, 2025, pp. 1–7.

[46] M. Shen, M. Umar, K. Maeng, G. E. Suh, and U. Gupta, “Hermes: Algorithm-system co-design for efficient retrieval-augmented generation at-scale,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 958–973.

[47] J. Shi, H. Zhang, Y. Wang, Q. Chen, Y. Shan, K. Fu, W. Wang, and M. Guo, “Elora: Efficient lora and kv cache management for multi-lora llm serving,” in *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2026, pp. 1–14.

[48] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, “Softermax: Hardware/software co-design of an efficient softmax for transformers,” in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 469–474.

[49] J. E. Stine, I. Castellanos, M. Wood, J. Henson, F. Love, W. R. Davis, P. D. Franzon, M. Bucher, S. Basavarajaiah, J. Oh, and R. Jenkal, “Freepdk: An open-source variation-aware design kit,” in *2007 IEEE International Conference on Microelectronic Systems Education (MSE’07)*. IEEE, 2007, pp. 173–174.

[50] Q. Su, W. Zhao, X. Li, M. Andoorveedu, C. Jiang, Z. Zhu, K. Song, C. Giannoula, and G. Pekhimenko, “Seesaw: High-throughput llm inference via model re-sharding,” *Proceedings of Machine Learning and Systems*, vol. 7, 2025.

[51] H. Sun, L.-W. Chang, W. Bao, S. Zheng, N. Zheng, X. Liu, H. Dong, Y. Chi, and B. Chen, “ShadowKV: KV cache in shadows for high-throughput long-context LLM inference,” in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 57 355–57 373.

[52] P. Sun, Y. Jiang, S. Chen, S. Zhang, B. Peng, P. Luo, and Z. Yuan, “Autoregressive model beats diffusion: Llama for scalable image generation,” *arXiv preprint arXiv:2406.06525*, 2024.

[53] H. Tang, Y. Wu, S. Yang, E. Xie, J. Chen, J. Chen, and S. Han, “Hart: Efficient visual generation with hybrid autoregressive transformer,” *arXiv preprint arXiv:2410.10812*, 2024.

[54] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, “Quest: query-aware sparsity for efficient long-context llm inference,” in *Proceedings of the 41st International Conference on Machine Learning*, 2024, pp. 47 901–47 911.

[55] T. Tang, K. Zhu, Q. Xu, Z. Jin, Y. Gu, Z. Zeng, R. Kadekodi, L. Zhao, A. Li, A. Krishnamurthy, and B. Kasikci, “Tactic: Adaptive sparse attention with clustering and distribution fitting for long-context LLMs,” in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: https://openreview.net/forum?id=tJod11fK1A

[56] K. Tian, Y. Jiang, Z. Yuan, B. Peng, and L. Wang, “Visual autoregressive modeling: Scalable image generation via next-scale prediction,” *Advances in neural information processing systems*, vol. 37, pp. 84 839–84 865, 2024.

[57] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, “Llama: Open and efficient foundation language models,” *arXiv preprint arXiv:2302.13971*, 2023.

[58] A. Voronov, D. Kuznedelev, M. Khoroshikh, V. Khrulkov, and D. Baranchuk, “Switti: Designing scale-wise transformers for text-to-image synthesis,” 2024.

[59] H. Wang, Z. Wang, Z. Yue, Y. Long, T. Wei, J. Yang, Y. Wang, C. Li, S. Wei, Y. Hu, and S. Yin, “Mcbp: A memory-compute efficient llm inference accelerator leveraging bit-slice-enabled sparsity and repetitiveness,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, 2025, pp. 1592–1608.

[60] X. Wang, Y. Zhuansun, C. Li, J. Wang, X. Hou, L. Sun, L. Wang, and M. Guo, “Asymserve: Demystifying and optimizing llm serving efficiency on cpu acceleration units,” in *International Symposium on Advanced Parallel Processing Technologies*. Springer, 2025, pp. 231–245.

[61] Y. Wang, H. Wang, and S. Q. Zhang, “Qsvd: Efficient low-rank approximation for unified query-key-value weight compression in low-precision vision-language models,” *arXiv preprint arXiv:2510.16292*, 2025.

[62] B. Xu, Y. Wang, Z. Wang, and P. Li, “AMS-KV: Adaptive KV caching in multi-scale visual autoregressive transformers,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 32, 2026, pp. 27 206–27 214.

[63] Z. Xu, D. Kong, J. Liu, J. Li, J. Hou, X. Dai, C. Li, S. Wei, Y. Hu, and S. Yin, “Wsc-llm: Efficient llm service and architecture co-exploration for wafer-scale chips,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 1–17.

[64] C. Yang, Y. Sui, J. Xiao, L. Huang, Y. Gong, C. Li, J. Yan, Y. Bai, P. Sadayappan, X. Hu, and B. Yuan, “Topv: Compatible token pruning with inference time optimization for fast and low-memory multimodal vision language model,” in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 19 803–19 813.

[65] H. Yang, Q. Huang, N. Pinckney, W. Turner, W. Zhou, Y. Zhang, C.-T. Ho, C.-C. Chang, and H. Ren, “Chipvqa: Benchmarking visual language models for chip design,” in *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 2025, pp. 1–6.

[66] Z. Yang, T. Xie, B. Lu, S. Liu, B. Yu, and M. Li, “Keep: A kv-cache-centric memory management system for efficient embodied planning,” *arXiv preprint arXiv:2602.23592*, 2026.

[67] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, “Flashinfer: Efficient and customizable attention engine for llm inference serving,” *Proceedings of Machine Learning and Systems*, vol. 7, 2025.

[68] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, “H2o: Heavy-hitter oracle for efficient generative inference of large language models,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 34 661–34 710, 2023.

[69] J. Zhou, H. Li, K. Jiang, Y. Sun, and T. Jia, “31.4 varsa: A visual autoregressive generation accelerator using performance-scalable multi-precision pe-lut and grid-similarity attention compression,” in *2026 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 69. IEEE, 2026, pp. 538–540.

[70] Z. Zhu, G. Li, T. Liu, Z. Mo, X. Liang, and J. Cheng, “Mata: A memory-efficient attention accelerator for llms exploiting look-back kv cache pruning,” *IEEE Transactions on Computers*, 2025.

[71] J. Zou, S. Wu, H. Duan, Q. Li, and C. J. Xue, “ContiguousKV: Accelerating LLM prefill with granularity-aligned KV cache management,” *arXiv preprint arXiv:2601.13631*, 2026.